

GovernorDelegator

Smart contract audit / ONE principal ledger

06 October 2026

REVIEWED SOURCE GovernorDelegator.sol / 61 lines / 2,569 bytes
SHA-256 df551a62e6b2b967645f51982aad65daedef35eef92d63cf3fbf187bbf9ffbe8

One low-severity finding.

Funded accounting and caller-only immediate exits hold under the specified ONE behavior. Vault-self destinations can permanently strand principal. Deployment and reward controls remain essential.

0 CRITICAL 0 HIGH 0 MEDIUM 1 LOW 1 INFO

FINDINGS & OBSERVATIONS

- L-01 Vault-self destinations permanently strand principal**
Deposits create unreachable vault-owned credits; withdrawals to the vault consume the caller's claim without releasing ONE. **Reject address(this) in all three paths.**
- I-01 Zero-value events do not prove economic activity**
Any zero-balance caller can emit a zero withdrawal. Zero deposits also emit. Indexers must use numeric deltas and debit Withdraw.caller.

INDEPENDENT VALIDATION

60 tests/properties passed. Invariant testing covered **2,560,000 actions** across 256 sequences, plus **32,768 accounting-sequence steps**, with no invariant failure.

Verified: funded batches, withdrawal authority, rollback and event accounting. Local mainnet-fork checks covered deployment, repeated batches and transaction encoding.

BUILD & DEPLOYMENT

Solidity 0.8.30 / Cancun / optimizer 200 / no CBOR. **2,186-byte runtime template.** Compiler settings and bytecode identity verified. ONE runtime checked at finalized block **26,130,360**.

Scope: GovernorDelegator and the specified ONE asset. L-01 is open. Production vaults, reserve Safe execution, allocations and reward implementation were not audited. Testing is not a formal proof.

Fable 5.1 Max, GPT-6 Astra Ultra, and Daybreak Blue were used for preparing and conducting this audit.
hiddenstate.xyz participates in OpenAI's Trusted Access for Cyber (TAC) and Anthropic's Cyber Verification Program.

Scope and source identity

GovernorDelegator is a custody ledger for Ethereum mainnet ONE. Deposits transfer tokens from the caller and credit a designated account. Each account can immediately withdraw its own credited principal to a chosen payout address. The governor can change the governor address and fee record, but cannot withdraw another account's principal.

Item	Reviewed identity
Source	GovernorDelegator.sol; 61 lines; 2,569 bytes
Source SHA-256	df551a62e6b2b967645f51982aad65daedef35eef92d63cf3fbf187bbf9ffbe8
Compiler	Solidity 0.8.30+commit.73712a01
Build settings	Optimizer 200; Cancun; legacy pipeline; no CBOR metadata
Supported asset	Ethereum mainnet ONE, 0xff74317e948695297acb79cb2de06111f5cc16b3

The review covers principal funding, withdrawals, governor authority, arithmetic, event accounting and token interaction. It includes source analysis, automated tests, invariant testing, compiler and bytecode verification, live asset-code checks and local mainnet-fork simulations.

The security conclusions depend on the specified ONE runtime: transfers move the exact amount, return Boolean success or revert, and have no callbacks, transfer fees, rebasing, blacklist, proxy or administrative transfer authority. The vault does not enforce the asset address in its constructor. Binding each deployment to the correct chain, token and bytecode is therefore essential.

Principal credits are nontransferable. The contract is not an ERC-20 receipt token or an ERC-4626 vault. It has no withdrawal queue, pause, rescue, upgrade or reward-distribution function. The fee is a configuration record and is not deducted from principal.

Scope limitations: production vault deployments, the reserve Safe configuration and execution, approved allocation records, frontend/indexer implementations and reward distribution were not audited. Local simulations do not establish production readiness. Testing and abstract-model checks are not a formal proof of the compiled contract.

Security findings

L-01 - Vault-self destinations permanently strand principal

Severity: Low. Status: open. Affected source: deposit lines 37-41, depositBatch lines 43-53, withdraw lines 56-59.

All three functions accept the vault itself as the credited account or payout destination. A deposit to the vault transfers real ONE into custody and assigns credit to `balanceOf[vault]`. The vault has no function that can withdraw on behalf of that account, making the credit unreachable.

A withdrawal to the vault produces a different form of loss. ONE's self-transfer leaves custody unchanged, but the vault subtracts the amount from the caller's claim. A holder with 100 units who calls `withdraw(30, vault)` retains only 70 units of claim while the vault still holds 100. The remaining 30 units have no recovery path.

Both cases were reproduced against the exact ONE runtime. The backing inequality still holds, so a solvency check alone does not detect the loss of redeemability. Exploitation requires a funder or withdrawing holder to select the bad destination; it does not let an attacker debit another holder's claim. A mistaken batch allocation can nevertheless strand a significant amount.

Recommendation: reject `delegator == address(this)` in `deposit`, the first `depositBatch` loop and `withdraw`. Validate batch recipients and confirm withdrawal destinations in clients as additional safeguards. Client-side checks alone do not prevent direct contract calls from reaching the vulnerable paths.

I-01 - Zero-value events do not prove economic activity

Severity: Informational. Status: documented. Affected source: lines 37-59.

A caller with no principal can execute `withdraw(0, validDestination)`. Zero deposits and zero-valued batch rows can also emit events without moving economic value. An empty batch performs a zero `transferFrom` and emits no per-entry `Deposit` events.

Indexers must use numeric deltas. A `Withdraw` event alone does not establish that the caller held positive principal or that funds were released. Zero/dust deposits from third parties must not reset another holder's earning history or count as activity. Positive-amount guards are optional event hygiene.

Severity assessment

The review identified **0 Critical, 0 High, 0 Medium, 1 Low and 1 Informational** findings. No path was identified for another holder or the governor to spend a victim's principal or the reserve's approval under the supported token behavior. Risks in deployment, transaction preparation and reward accounting are addressed in the requirements below.

Validation results

60 tests/properties passed. Four fuzz properties ran 2,048 cases, including 32,768 steps of accounting sequences. The invariant campaign checked **2,560,000 actions** across 256 sequences, with no invariant failure, handler revert or discarded action.

The tests compared expected balances and event-derived balances with contract storage, checked custody backing and failure rollback, and exercised full withdrawals for simulated authorized holders. Incompatible-token tests established the limits of the supported asset assumption. Tests do not establish ownership of production keys or an exit path for every possible recipient contract.

Let $B[a]$ denote an account's principal, P the sum of all principal credits and C the ONE balance of the vault.

Property	Result	Evidence and qualification
Custody backing: $C \geq P$	Pass	Deposits and withdrawals preserve backing; donations and vault-self withdrawal surplus are accounted for. Backing does not imply every credit is redeemable.
Funded deposits	Pass	Deposits debit the caller. Each batch pulls its summed amount once; duplicate recipients receive additive credits.
Failure atomicity	Pass	Failed validation, insufficient funds/allowance, arithmetic overflow and token failures preserve balances and allowances.
Withdrawal authority	Pass	Only the caller's principal is debited. Overdraw reverts, including for the governor.
Governor and fee isolation	Pass	Configuration calls do not change principal. Stored fee remains between 0 and 10,000 bps.
No administrative credit minting	Pass	Only funded deposit functions increase principal; no fallback or alternative credit path exists.
Event reconstruction	Pass	Ordered events reconstruct principal, governor and fee. Withdrawals debit the event caller, not the payout destination.
Immediate exits	Qualified pass	Full exits succeed for callable holders under exact ONE and sufficient backing. Recipient control remains a separate requirement.
Recipient reachability	Exception	Vault-owned credit is unreachable (L-01). Other contracts may also lack the ability to call withdraw.
Reentrancy exposure	Conditional	The specified ONE transfer paths have no external callbacks. Different token behavior invalidates this conclusion.
Same-block holdings	Pass	Deposit and withdrawal can occur with the same timestamp; their holding duration is zero.

Static analysis with Slither 0.11.6 produced no additional applicable vulnerability after review. Eleven arithmetic and mapping obligations were verified in an abstract Z3 model, including caller-only debit and non-deposit non-increase. Those results depend on the model's translation and exact-transfer assumptions; no source-level or bytecode formal proof is claimed.

Build and contract interface

The reviewed compiler configuration reproduced the ABI, creation code, runtime template and storage layout. Deployment verification must use the runtime with the asset immutable populated, rather than the zero-placeholder template.

Build item	Verified value
Creation template	2,463 bytes, before two constructor words
Creation keccak256	0x72dad98e0eda48dc4d8aae1760b3a11b664c7005dd13f6719d2209355128d4f2
Runtime template	2,186 bytes
Runtime template keccak256	0x914d3202528eb35450bf1d9582ebb8e83363c4af2b4ea5f81621c866fa372bea
Asset immutable	32-byte fields at offsets 272, 481, 845 and 1361; address left-padded with zeros
Runtime for mainnet ONE	0xc1f4cf5c480968601dde8f0e72521acfa7948d4d6b668dfa8f446f81375a47e6
Storage	governor slot 0; balanceOf mapping anchor 1; feeBps slot 2
Metadata	bytecodeHash = none; appendCBOR = false

Function	Selector	Meaning
transferGovernor(address)	b8cc9ce6	Governor-only immediate role change
setFeeBps(uint16)	023b1fc9	Governor-only 0..10,000 fee record
deposit(uint256,address)	6e553f65	Caller funds delegator; no return value
depositBatch(uint256[],address[])	c2fff781	Amounts first; one caller-funded transfer
withdraw(uint256,address)	00f714ce	Debit caller; pay second argument
asset()	38d52e0f	Immutable token address
balanceOf(address)	70a08231	Nontransferable principal credit
feeBps()	24a9d853	Current fee record
governor()	0c340a24	Current configuration authority

The contract has no receive or fallback function. Its four events are GovernorTransferred, FeeUpdated, Deposit and Withdraw. Deposit and Withdraw index caller and delegator; assets is an unindexed uint256. The Withdraw topic is 0x9b1bfa7fa9ee420a16e124f794c35ac9f90472acc99140eb2f6447c714cad8eb.

Integration requirements

Withdraw(uint256,address) debits the caller and pays the second argument. If Alice withdraws to Bob, Alice's principal decreases and Bob receives ONE; Bob's vault credit is unchanged. Reward and principal accounting must follow the debited caller.

Batch calldata must encode amounts before delegators. Reversing these arrays can remain ABI-valid and, for some values, fund unintended addresses. Independently decode each proposed transaction and compare every amount and recipient with the approved allocation before signing. Do not rely on insufficient funding to catch an encoding error.

The balanceOf, asset and deposit selectors overlap familiar standards, but this contract has no transferable shares. An ERC-4626 caller expecting a shares return value from deposit fails return decoding. Use the contract's actual ABI. Clients should confirm payout addresses, reject zero/vault destinations and explain checked-subtraction Panic(0x11) as insufficient principal when applicable.

Compiler and source verification

The compiler bug registry contains seven entries whose version ranges include 0.8.30; none applies to this source and configuration. The relevant triggering features are absent. [Official Solidity bug registry](#).

Public source verification should use the exact standard JSON settings and constructor arguments. Etherscan requires standard JSON for nondefault metadata-hash settings. This metadata-free build does not provide an embedded source fingerprint for Sourcify's exact-match assurance. Public explorer verification was not assessed for a production vault. [Etherscan verification guidance](#), [Sourcify matching definitions](#).

Batch gas

Recipients / operation	Default isolation	Isolation disabled
100 fresh	2,652,374	2,562,430
200 fresh	5,247,759	5,098,807
400 fresh	10,442,844	10,175,780
500 fresh	13,042,520	12,716,376
600 fresh	15,643,591	15,258,379
400 top-up	3,571,813	1,379,949
Partial withdraw	62,180	26,652
Full withdraw	36,173	6,752

These figures are call-gas measurements in Foundry 1.8.1, not Safe transaction costs. Storage warmth and slot-clearing make these measurements state-dependent. Add calldata, intrinsic gas and Safe/MultiSend overhead when sizing transactions. Use **at most 400 fresh recipients as a planning ceiling**, subject to simulation of the actual Safe transaction with margin below the 16,777,216 gas cap. A production 400-entry Safe transaction was not validated. [EIP-7825](#).

Deployment and operating requirements

Asset and deployment binding

At finalized Ethereum block **26,130,360**, two RPC providers returned the same 1,465-byte ONE runtime, keccak256 0x277f937d07613df41830d132315454a36da1eb96d4b07e65998141ebc32613ba. The block hash was 0x123a68c75c294eafa5c5ceb89cc8098e3cb68a357b8c31612a1220749a5afe66. This observation was made on 06 October 2026.

Local mainnet-fork deployment checks passed and rejected wrong-chain, wrong-asset and wrong-governor configurations. These checks exercised direct CREATE deployments with simulated accounts; they do not verify a production vault or governor's key control.

Before funding, verify chain ID 1, the exact ONE address and runtime, the populated vault runtime, constructor arguments, canonical creation receipt, initial governor and zero initial fee. Confirm the governor's ability to call the contract and record the vault-to-validator association. A zero mapping anchor alone does not prove that every principal entry is empty. Factory or CREATE2 deployment requires verification of that deployment path as well.

Funding and reconciliation

Successful batches are additive, including identical repeats. Local fork tests confirmed exact aggregate funding, credited balances, zero residual allowance and the effect of repeating a successful batch.

Transaction-preparation checks rejected incorrect allocations and swapped array encodings.

Production funding requires independently approved allocations and evidence that each recipient can withdraw on Ethereum. Reject zero amounts, zero/vault recipients and accounts without a demonstrated exit path. Deposits are permissionless; a funded credit is not proof of recipient identity or control.

Approve the exact batch sum and execute depositBatch in a reviewed Safe MultiSend transaction. Simulate the complete transaction using the actual Safe configuration, then reconcile token funding, Deposit count and sum, individual credits, custody and remaining allowance. The reserve Safe, its modules/guards/signers and production allocations were not validated by this review.

Before retrying a batch, reconcile its allocation content, calldata, Safe nonce and transaction hash against canonical execution. A different transaction hash or nonce does not make a repeated allocation safe. Mark completion only after the required finality and reverse provisional accounting after a reorganization.

Governance and monitoring

Governor transfers take effect immediately. A destination that cannot call the vault can permanently freeze fee configuration, although holders retain their withdrawal authority. Verify destination identity and control before each handover. The fee can range from 0 to 100%; it does not alter principal, but reward systems must apply it under explicit historical rules.

Monitor custody against event-reconstructed principal. The surplus $C - \text{sum}(\text{Deposit.assets}) + \text{sum}(\text{Withdraw.assets})$ must remain nonnegative under supported behavior. Both donations and vault-self withdrawals can increase surplus; a vault-self deposit may instead trap principal without creating a surplus. Alert separately on vault-self destinations, unexpected role/fee changes and accounting mismatches.

There is no pause, rescue or upgrade mechanism. Operators can stop new funding, but cannot freeze holders, reverse mistaken payouts or recover stranded credits through the vault.

Reward accounting requirements

Reward calculation and distribution are outside the contract scope. Immediate withdrawals permit entry and exit around snapshots. Reward systems must use balance history and the fee and governor applicable when earnings accrue.

Accrue eligible balances over defined intervals using canonical block, transaction and log order. Deposit credits its delegator; Withdraw debits its caller. Equal timestamps add no duration. Preserve earnings after exit, exclude periods before a deposit, and prevent zero events or third-party deposits from resetting another holder's earning history.

Illustrative time-weighted policy

For a fixed interval budget R , let W be total token-seconds. Let $N[a]$ integrate each holder's balance multiplied by one minus the fee fraction applicable at that time. Let $K[g]$ integrate total balance multiplied by that fee while g is governor. Paying $R \cdot N[a] / W$ to holders and $R \cdot K[g] / W$ to historical governors conserves the budget. A zero-weight interval needs an explicit unallocated-budget rule.

This is an illustration, not an approved production policy or proof of Harmony EPoS equivalence. Raw token-seconds and contemporaneous stake shares can produce different rewards. Effective-stake caps, election thresholds, median rules, changing denominators, reward rates and commission constraints must be specified before claiming EPoS replication.

Example	Inputs	Result under raw token-seconds
Same-block round trip	Deposit and withdraw 900 at the same timestamp	Zero earning weight.
Hold around a snapshot	B holds 100 for 100 seconds; A holds 900 for two seconds around the snapshot	A has 90% at the snapshot but 1,800 of 11,800 token-seconds. A 1,180-unit budget pays A 180 and B 1,000.
Fee and governor history	100 staked for 100 seconds; 10% fee; governor changes halfway; fee rises to 100% for the final second	A 1,000-unit budget pays the holder 891, the first governor 50 and the second 59.
Exit to another address	A holds 100 for 40 seconds and withdraws to C; B holds 100 for the next 60 seconds	A retains 40% of the reward; B earns 60%. C receives the principal payout, not A's earning identity.
Deposit and top-up	100 credited at second 20, then 50 at second 60 of a 100-second interval	10,000 token-seconds; no credit before second 20.
Different economic policies	A holds 100 throughout; B adds 100 halfway; budget 600	Raw token-seconds pays 400/200. Uniform emission by instantaneous stake share pays 450/150.

Before rewards launch, define eligibility, budgets, intervals, historical commission, migration dates, rounding, finality and reorganization handling. Backdating requires authenticated allocations and an explicit rule. Audit stablecoin distribution and claims separately.

Appendix – reviewed source

Original source lines 1-42.

```
// SPDX-License-Identifier: MIT
// ONE governor/delegator vault with funded batch deposits; see LICENSE and NOTICE.txt.
pragma solidity ^0.8.24;
interface IERC20 {
    function transfer(address to, uint256 value) external returns (bool);
    function transferFrom(address from, address to, uint256 value) external returns (bool);
}
contract GovernorDelegator {
    IERC20 public immutable asset;
    address public governor;
    mapping(address => uint256) public balanceOf;
    uint16 public feeBps;
    event GovernorTransferred(address indexed previous, address indexed next);
    event FeeUpdated(uint16 previousFeeBps, uint16 newFeeBps);
    event Deposit(address indexed caller, address indexed delegator, uint256 assets);
    event Withdraw(address indexed caller, address indexed delegator, uint256 assets);
    modifier onlyGovernor() {
        require(msg.sender == governor);
        _;
    }
    constructor(IERC20 token, address initialGovernor) {
        require(initialGovernor != address(0));
        governor = initialGovernor;
        emit GovernorTransferred(address(0), initialGovernor);
        asset = token;
    }
    function transferGovernor(address next) external onlyGovernor {
        require(next != address(0));
        emit GovernorTransferred(governor, next);
        governor = next;
    }
    function setFeeBps(uint16 value) external onlyGovernor {
        require(value <= 10_000);
        emit FeeUpdated(feeBps, value);
        feeBps = value;
    }
    function deposit(uint256 amount, address delegator) external {
        require(delegator != address(0));
        require(asset.transferFrom(msg.sender, address(this), amount));
        balanceOf[delegator] += amount;
        emit Deposit(msg.sender, delegator, amount);
    }
}
```

Reviewed source (continued)

Original source lines 43-61.

```
function depositBatch(uint256[] calldata amounts, address[] calldata delegators) external {
    require(delegators.length == amounts.length);
    uint256 total;
    for (uint256 i; i < delegators.length; ++i) {
        require(delegators[i] != address(0));
        total += amounts[i];
    }
    require(asset.transferFrom(msg.sender, address(this), total));
    for (uint256 i; i < delegators.length; ++i) {
        balanceOf[delegators[i]] += amounts[i];
        emit Deposit(msg.sender, delegators[i], amounts[i]);
    }
}
function withdraw(uint256 amount, address delegator) external {
    balanceOf[msg.sender] -= amount;
    require(asset.transfer(delegator, amount));
    emit Withdraw(msg.sender, delegator, amount);
}
```